

<lista-modelos>

- Modelo 1: Gemini 3 Pro
- Modelo 2: Qwen3 Max
- Modelo 3: DeepSeek V3.2 Reasoner
- Modelo 4: GPT-5.1
- Modelo 5: Claude 4.5 Sonnet
- Modelo 6: Kimi K2.5

</lista-modelos>

<pedidos>

<pedido1>

De acordo com fontes confiáveis e com as melhores práticas de desenvolvimento assistido por IA, pense em quais princípios um humano teria que levar em conta ao gerar código com IA para que o código saia com a melhor qualidade - independente do tipo de software.

</pedido1>

<pedido2>

Com base nesses princípios que você definiu, prepare um template de prompt inicial para gerar código - independente de software.

</pedido2>

</pedidos>

—

<respostas-modelos>

<modelo1>

<resposta1>

Excelente pergunta sobre os princípios para gerar código de alta qualidade com IA! Vou preparar uma resposta detalhada abordando as melhores práticas e considerações humanas essenciais para garantir a excelência do software.

Com base nas melhores práticas da engenharia de software moderna e na evolução do desenvolvimento assistido por IA (AI-Assisted Development), compilei os princípios fundamentais que um desenvolvedor humano deve adotar.

A premissa central é: **a IA não é o piloto, é o motor.** A qualidade do código é diretamente proporcional à qualidade da direção (contexto e supervisão) que você fornece.

Aqui estão os **7 Princípios de Ouro** para gerar código de alta qualidade com IA:

1. O Princípio do Contexto Rico (Context is King) 🏰

A IA não tem memória de longo prazo do seu projeto, a menos que você a forneça. Um prompt vago gera código genérico.

- **O que fazer:** Antes de pedir o código, defina o cenário. Especifique a linguagem, o framework (com versões), as bibliotecas utilizadas e o objetivo de negócio.

- **Exemplo:** Em vez de *"Crie uma função de login"*, use *"Atuando como um Engenheiro Sênior em Python/Django, crie uma função de login que utilize JWT para autenticação, respeite as normas da PEP-8 e trate exceções de banco de dados especificamente."*

2. O Princípio da Decomposição (Dividir para Conquistar) 🧩

Modelos de IA (LLMs) perdem coerência quando tentam gerar arquivos gigantescos ou sistemas complexos de uma só vez.

- **A Prática:** Quebre o problema em unidades lógicas menores. Gere interfaces primeiro, depois a implementação de funções individuais, depois os testes.
- **Benefício:** Isso facilita a depuração e garante que a IA mantenha o foco na lógica específica daquela pequena parte, reduzindo alucinações.

3. O Princípio da Desconfiança Saudável (Trust but Verify) 🤖

Nunca aceite o primeiro output como verdade absoluta. A IA pode alucinar métodos que não existem, usar bibliotecas depreciadas ou criar falhas lógicas sutis.

- **A Regra:** Você é o **Code Reviewer**. Leia cada linha. Se você não entende o que o código faz, não o coloque em produção. Peça para a IA explicar a lógica linha a linha se necessário.

4. O Princípio da Segurança Primeiro (Security by Design) 🛡️

A IA é treinada em todo tipo de código da internet, incluindo código inseguro. Ela pode sugerir injeção de SQL, expor chaves de API ou usar criptografia fraca se não for guiada.

- **Ação:** Instrua explicitamente a IA a seguir práticas de segurança (ex: *"Sanitize todos os inputs do usuário para evitar XSS"*). Nunca cole chaves de API reais ou dados sensíveis de clientes no prompt.

5. O Princípio da Iteração Dialética ↺

O primeiro rascunho raramente é o melhor. Trate a IA como um par de programação (Pair Programmer).

- **O Processo:**
 1. Gere o código.
 2. Aponte erros ou melhorias (*"Isso parece ineficiente, podemos otimizar a complexidade de tempo?"* ou *"Reescreva para ser mais legível"*).
 3. Refine até atingir o padrão desejado.
- **Dica:** Peça para a IA criticar o próprio código antes de finalizar. *"Quais são as potenciais falhas ou edge cases neste código que você acabou de gerar?"*

6. O Princípio dos Testes Simultâneos

Código sem teste é dívida técnica imediata. A IA é excelente em escrever testes unitários repetitivos.

- **A Prática:** Para cada bloco de código funcional gerado, exija imediatamente a geração dos testes unitários correspondentes (ex: Jest, Pytest, JUnit). Isso valida se o código gerado realmente faz o que a IA diz que faz.

7. O Princípio da Manutenibilidade e Estilo

Se o código gerado não segue o padrão do seu projeto, ele se torna um "corpo estranho" difícil de manter.

- **Ação:** Forneça exemplos do seu código existente (one-shot prompting) para que a IA mimetize seu estilo de nomenclatura, indentação e estrutura de comentários. Peça explicitamente por **Type Hinting** (tipagem forte/sugerida) e **Docstrings** para garantir que o código seja legível por humanos no futuro.

Resumindo a Mentalidade

Para obter qualidade máxima, o humano deixa de ser apenas um "digitador de código" e passa a atuar como um **Arquiteto de Software e Revisor Técnico**.

A fórmula da qualidade: (Contexto Claro + Decomposição do Problema) x (Revisão Humana Rigorosa) = Código de Excelência.

</resposta1>

<resposta2>

Com certeza! Prepararei um template de prompt inicial para geração de código, focado nos princípios que discutimos. Ele será versátil e aplicável a diferentes contextos de desenvolvimento de software.

Excelente iniciativa. Transformar princípios abstratos em uma ferramenta prática (o prompt) é o que diferencia um usuário comum de um "Engenheiro de Prompt" eficaz.

Abaixo, criei um **"Master Prompt"** estruturado. Ele foi desenhado para ser modular: você preenche as lacunas entre colchetes [. . .] e garante que a IA tenha todas as restrições necessárias antes de escrever a primeira linha de código.

Template de Prompt: O "Arquiteto de Código"

Copie e preencha os campos abaixo:

1. PERSONA E CONTEXTO (Princípio do Contexto Rico)

Atue como um Engenheiro de Software Sênior e Especialista em Segurança.

Estou desenvolvendo um software para [DESCREVA O OBJETIVO DO PROJETO - ex: um e-commerce de nicho, um sistema de gestão hospitalar, um script de automação de dados].

O objetivo de negócio desta funcionalidade específica é [DESCREVA O OBJETIVO - ex: permitir que usuários recuperem senha de forma segura].

2. STACK TECNOLÓGICA (Princípio do Contexto Rico)

- Linguagem: [EX: Python 3.11, TypeScript 5.0, Go 1.20]
- Frameworks/Libs: [EX: Django 4.2, React 18, Pandas, AWS SDK]
- Banco de Dados (se aplicável): [EX: PostgreSQL, MongoDB, Redis]
- Estilo de Código: [EX: PEP-8, Google Style Guide, Airbnb Style Guide]

3. TAREFA ESPECÍFICA (Princípio da Decomposição)

Não gere o sistema todo. Foque APENAS na seguinte tarefa:

[DESCREVA A TAREFA ATÔMICA - ex: Crie a função de validação de token JWT no backend ou Crie o componente visual do botão de checkout].

Entrada esperada: [O que a função/componente recebe?]

Saída esperada: [O que deve retornar?]

4. REQUISITOS DE QUALIDADE E SEGURANÇA (Princípios de Segurança e Desconfiança)

O código gerado DEVE seguir estritamente estas regras:

1. Segurança: Implemente proteções contra [EX: SQL Injection, XSS, Race Conditions] e nunca exponha dados sensíveis.
2. Robustez: Trate erros e exceções (Edge Cases) explicitamente. Não deixe falhas silenciosas.
3. Performance: Evite complexidade desnecessária ($O(n^2)$ ou pior) se houver alternativa mais eficiente.
4. Princípios: Siga SOLID, DRY (Don't Repeat Yourself) e KISS (Keep It Simple, Stupid).

5. DOCUMENTAÇÃO E MANUTENIBILIDADE (Princípio da Manutenibilidade)

- Adicione Docstrings/Comentários explicativos para lógicas complexas (mas evite comentar o óbvio).
- Use nomes de variáveis e funções semânticos e em [IDIOMA - ex: Inglês ou Português].
- Utilize Tipagem Estática/Type Hinting onde a linguagem permitir.

6. TESTES (Princípio dos Testes Simultâneos)

Após gerar o código, crie também:

- Casos de teste unitários (usando [FRAMEWORK DE TESTE - ex: Pytest, Jest]) cobrindo o caminho feliz e pelo menos 2 casos de erro/borda.

7. FORMATO DE SAÍDA

Apresente a solução na seguinte ordem:

1. Breve explicação da abordagem lógica.
 2. Bloco de código da implementação.
 3. Bloco de código dos testes.
-



Por que este template funciona? (A Lógica por trás)

1. **Persona Sênior:** Ao pedir para a IA atuar como "Sênior", você ativa pesos no modelo associados a códigos mais técnicos, seguros e bem estruturados, evitando soluções de tutoriais básicos.
2. **Delimitação de Escopo (Seção 3):** Ao forçar a definição de "Entrada" e "Saída", você impede que a IA alucine funcionalidades que você não pediu.
3. **Regras Explícitas (Seção 4):** A IA tende a ser preguiçosa. Se você não pedir tratamento de erros (*try/catch*), ela muitas vezes não fará. Este prompt *exige* robustez.
4. **Testes Obrigatórios (Seção 6):** Isso força a IA a pensar em código testável. Se o código for difícil de testar, a IA muitas vezes o reescreve internamente para torná-lo mais modular antes de apresentar a resposta.

Como usar na prática:

Salve este template no seu bloco de notas (ou ferramenta de snippets). Quando for codificar, gaste 2 minutos preenchendo as lacunas. Esses 2 minutos economizarão 20 minutos de refatoração e correção de bugs depois.

</resposta2>

</modelo1>

—

<modelo2>

<resposta1>

Excelente pergunta sobre os princípios para gerar código de alta qualidade com IA! Vou preparar uma resposta detalhada abordando as melhores práticas e considerações humanas essenciais para garantir a excelência do software.

Princípios fundamentais para gerar código de alta qualidade com IA, baseados em fontes confiáveis e melhores práticas de desenvolvimento, incluem:



1. Especificação Clara e Contextualizada

- **Defina requisitos precisos:** Descreva exatamente o que o código deve fazer, incluindo casos de uso, entradas/saídas esperadas e comportamentos em cenários extremos
- **Forneça contexto do domínio:** Explique o contexto de negócio ou técnico para que a IA compreenda as implicações das decisões de implementação

- **Especifique restrições técnicas:** Versões de linguagem, bibliotecas permitidas, padrões de arquitetura e requisitos de desempenho



2. Segurança e Confiabilidade como Prioridade

- **Validação rigorosa de entradas:** Sempre solicite validação de dados e tratamento de exceções adequado
- **Evite hardcoded secrets:** Nunca permita que credenciais, chaves API ou dados sensíveis sejam embutidos no código
- **Siga princípios de segurança:** Princípio do menor privilégio, defesa em profundidade e sanitização de dados



3. Testabilidade e Verificação

- **Exija testes automatizados:** Peça à IA para gerar testes unitários, de integração e casos de teste abrangentes
- **Verifique manualmente o código:** Revise criticamente toda a saída da IA, especialmente lógica complexa e manipulação de dados
- **Teste em ambientes isolados:** Execute o código em ambientes controlados antes de qualquer uso em produção



4. Manutenibilidade e Legibilidade

- **Siga convenções de código:** Adote os padrões de estilo e nomenclatura da linguagem e do time
- **Documente adequadamente:** Inclua comentários explicativos para lógica complexa e documentação de APIs públicas
- **Mantenha funções focadas:** Cada função/método deve ter uma única responsabilidade e ser facilmente compreensível



5. Integração com Fluxo de Trabalho Existente

- **Alinhe com arquitetura existente:** O código gerado deve seguir os padrões e decisões arquiteturais do projeto atual
- **Considere dependências:** Avalie cuidadosamente novas dependências sugeridas pela IA quanto a segurança, licenciamento e manutenção
- **Versionamento adequado:** Trate o código gerado por IA como qualquer outro código - versione, revise e integre conforme o fluxo normal



6. Consciência das Limitações da IA

- **Não confie cegamente:** A IA pode gerar código plausível mas incorreto, inseguro ou ineficiente
- **Verifique referências e APIs:** Confirme que bibliotecas, métodos e APIs mencionados existem e funcionam como descrito
- **Atualize conhecimento:** Mantenha-se atualizado sobre as capacidades e limitações dos modelos de IA que utiliza



7. Ética e Responsabilidade

- **Respeite licenças:** Verifique que o código gerado não viola direitos autorais ou licenças de software
- **Atribuição adequada:** Quando baseado em exemplos específicos, dê crédito apropriado
- **Responsabilidade final:** Lembre-se que **você** é responsável pelo código que produz, mesmo quando gerado com assistência de IA

Lembrete crucial: A IA é uma ferramenta poderosa de **aumento de produtividade**, não um substituto para o julgamento humano, experiência técnica e responsabilidade profissional. O melhor código gerado com IA resulta da **colaboração inteligente** entre desenvolvedor experiente e assistente artificial.

</resposta1>

<resposta2>

Entendido! Prepararei um template de prompt inicial focado em gerar código, seguindo os princípios que definimos. Você receberá um modelo versátil e eficaz para suas necessidades de desenvolvimento.

Template Abrangente para Geração de Código de Alta Qualidade com Assistência de IA

Este template foi projetado para guiar desenvolvedores na criação de prompts detalhados e eficazes para ferramentas de IA, garantindo que o código gerado seja robusto, seguro, testável, manutenível e alinhado com as melhores práticas. Preencha cada seção com o máximo de detalhes possível.

Instruções para a IA:

Gere o código conforme as especificações detalhadas abaixo. Priorize clareza, segurança, testabilidade e manutenibilidade. Inclua comentários explicativos para lógica complexa e exemplos de uso, se aplicável.

1. Contexto e Requisitos do Projeto

- **Objetivo Geral/Propósito:**
[Descreva o problema que o código deve resolver e o objetivo final. Ex: "Desenvolver um módulo para processar pagamentos online, integrando-se com um gateway de terceiros para garantir transações seguras e eficientes."]
- **Funcionalidade Principal Requerida:**
[Liste as funcionalidades essenciais que o código deve implementar. Ex: "1. Iniciar transação de pagamento. 2. Confirmar pagamento. 3. Lidar com falhas de pagamento. 4. Registrar histórico de transações."]
- **Entradas Esperadas:**
[Detalhe os tipos de dados, formatos e fontes das entradas. Ex: "Objeto JSON contendo 'valor' (float), 'moeda' (string, ex: 'BRL'), 'id_cliente' (string), 'token_cartao' (string)."]
- **Saídas Esperadas:**
[Detalhe os tipos de dados, formatos e destinos das saídas. Ex: "Objeto JSON com 'status' (string: 'sucesso'/'falha'), 'id_transacao' (string), 'mensagem_erro' (string, se aplicável)."]
- **Comportamento Desejado (Casos de Uso):**
[Descreva o fluxo esperado para cenários comuns e de sucesso. Ex: "Usuário envia dados de pagamento -> Sistema valida -> Envia para gateway -> Recebe confirmação -> Atualiza status no DB -> Retorna sucesso."]
- **Comportamento em Casos de Borda/Exceção:**
[Descreva como o código deve lidar com entradas inválidas, falhas de serviço externo, timeouts, etc. Ex: "Se o gateway retornar erro, registrar o erro, tentar novamente (se apropriado) e retornar status de falha com mensagem específica."]
- **Expectativas de Desempenho:**
[Especifique requisitos de latência, throughput ou uso de recursos. Ex: "Processar transações em menos de 500ms. Suportar 100 transações/segundo."]

2. Especificações e Restrições Técnicas

- **Linguagem de Programação e Versão:**
[Ex: "Python 3.9", "TypeScript 4.x", "Java 17"]
- **Frameworks/Bibliotecas e Versões:**
[Ex: "Django 4.x", "React 18", "Spring Boot 3.x", "Numpy 1.22"]
- **Padrões de Arquitetura/Design Existentes:**
[Ex: "Seguir arquitetura de microsserviços", "Utilizar padrão Repository", "Injeção de Dependência."]
- **Plataforma/Ambiente de Execução:**
[Ex: "Servidor Linux", "Ambiente AWS Lambda", "Navegador Web (frontend)"]

- **Restrições Conhecidas/Considerações Específicas:**
[Ex: "Não usar bibliotecas externas não aprovadas", "Evitar recursão profunda", "Compatibilidade com IE11 (se aplicável)."]

3. Requisitos de Segurança e Validação

- **Regras de Validação de Entrada:**
[Ex: "Todos os campos de entrada devem ser validados contra injeção SQL/XSS", "Valores numéricos devem estar dentro de um intervalo específico", "Strings devem ter comprimento máximo."]
- **Necessidades de Sanitização de Dados:**
[Ex: "Remover tags HTML de entradas de usuário", "Normalizar strings para UTF-8."]
- **Contexto de Autenticação/Autorização:**
[Ex: "O código será executado sob um usuário autenticado com permissões 'admin'", "Verificar tokens JWT para autorização."]
- **Tratamento de Dados Sensíveis:**
[Ex: "Dados de cartão de crédito não devem ser armazenados localmente", "Senhas devem ser hashadas com bcrypt", "Usar variáveis de ambiente para chaves API."]
- **Melhores Práticas de Segurança a Seguir:**
[Ex: "Princípio do menor privilégio", "Tratamento seguro de erros para evitar vazamento de informações", "Prevenção de ataques CSRF."]

4. Expectativas de Teste

- **Tipos de Testes Requeridos:**
[Ex: "Testes unitários para todas as funções/métodos", "Testes de integração para a comunicação com o gateway de pagamento."]
- **Metas de Cobertura de Código:**
[Ex: "Mínimo de 80% de cobertura de linha para testes unitários."]
- **Casos de Teste/Cenários Específicos:**
[Forneça exemplos concretos de entradas e saídas esperadas para testes. Ex: "Teste unitário para processar_pagamento(valor=100, moeda='BRL') deve retornar sucesso. Teste para processar_pagamento(valor=-10) deve lançar ValueError."]
- **Requisitos de Mocking/Stubbing:**
[Ex: "Mocar chamadas externas ao gateway de pagamento para testes unitários", "Stubbar o banco de dados para testes de integração."]

5. Qualidade e Estilo do Código

- **Guia de Estilo de Codificação:**
[Ex: "Seguir PEP 8 para Python", "Usar o guia de estilo do Google para Java", "Formatação com Prettier."]

- **Convenções de Nomenclatura:**
[Ex: "Variáveis em snake_case, classes em CamelCase", "Métodos privados prefixados com _."]
- **Requisitos de Documentação:**
[Ex: "Docstrings para todas as funções públicas", "Comentários inline para lógica complexa", "Gerar documentação Javadoc/Sphinx."]
- **Metas de Modularidade/Reusabilidade:**
[Ex: "O código deve ser modular e facilmente reutilizável em outros projetos", "Separar preocupações em classes/módulos distintos."]
- **Expectativas de Legibilidade/Clareza:**
[Ex: "Código autoexplicativo, com o mínimo de comentários possível", "Evitar abreviações obscuras."]
- **Restrições de Complexidade:**
[Ex: "Funções não devem exceder 50 linhas", "Complexidade ciclomática máxima de 10 por função."]

6. Considerações de Integração

- **Dependências (Internas/Externas):**
[Liste quaisquer módulos internos ou bibliotecas externas que o código precisará interagir. Ex: "Depende do módulo auth_service para autenticação", "Utiliza a biblioteca requests para chamadas HTTP."]
- **Endpoints/Interfaces de API:**
[Descreva como o código se integrará com outras APIs. Ex: "Consumir API REST em https://api.gateway.com/v1/payments", "Expor endpoint /api/v1/process_payment."]
- **Interações com Armazenamento de Dados/Banco de Dados:**
[Ex: "Interagir com PostgreSQL usando SQLAlchemy ORM", "Armazenar dados em um bucket S3."]
- **Relatórios de Erros/Logging:**
[Ex: "Usar o sistema de logging padrão do framework (ex: logging em Python)", "Registrar erros críticos no Sentry."]
- **Ambiente de Implantação/CI/CD:**
[Ex: "O código será implantado via Docker em Kubernetes", "Deve ser compatível com pipeline de CI/CD existente (Jenkins/GitLab CI)."]
- **Licenciamento/Atribuição (se aplicável):**
[Ex: "O código deve ser compatível com licença MIT", "Se usar trechos de código de terceiros, incluir atribuição."]

</resposta2>

</modelo2>

—

<modelo3>

<resposta1>

Entendido! Vou preparar uma resposta detalhada sobre os princípios essenciais que um humano deve considerar ao usar IA para gerar código, garantindo a máxima qualidade em qualquer tipo de software.

De acordo com as melhores práticas de desenvolvimento assistido por IA e pesquisas do setor, existem princípios fundamentais que um desenvolvedor deve considerar para garantir que o código gerado por IA atinja a **maior qualidade possível**, independentemente do tipo de software:

1. Especificidade Intencional

A IA funciona melhor com **instruções claras, concretas e contextualizadas**. Em vez de pedir "crie uma função", especifique:

- **Propósito exato** da funcionalidade
- **Condições de entrada e saída** esperadas
- **Casos de borda** que devem ser tratados
- **Restrições de performance** ou segurança

Exemplo prático:

 "Faça um login"

 "Crie uma função de autenticação que valide email no formato válido, senha com mínimo 8 caracteres (letra, número, símbolo), retorne token JWT com expiração de 24h e registre tentativas falhas para prevenção de brute-force."

2. Contextualização Estratégica

Forneça **contexto arquitetural** para que a IA gere código que se integre organicamente:

- **Padrões do projeto** já estabelecidos (MVC, Clean Architecture, etc.)
- **Bibliotecas e frameworks** em uso
- **Convenções de nomenclatura** da equipe
- **Restrições técnicas** do ambiente (versões de linguagem, banco de dados, etc.)

Princípio: A IA não adivinha seu contexto – você precisa explicitá-lo.



3. Mentalidade de Revisor Crítico

Trate o código gerado como um **rascunho de um estagiário talentoso**:

- **Revise linha por linha** antes de integrar
 - **Teste imediatamente** com casos representativos
 - **Verifique vulnerabilidades** de segurança comuns (SQL injection, XSS, etc.)
 - **Avalie a legibilidade** – se você não entende, outros também não entenderão
-



4. Abordagem Modular e Iterativa

Gere código em **blocos gerenciáveis**, não sistemas completos de uma vez:

1. **Comece com interfaces/contratos**
2. **Implemente funcionalidade por funcionalidade**
3. **Integre e teste após cada ciclo**
4. **Refatore com base nos resultados**

Benefício: Identifica problemas cedo e mantém o controle sobre a qualidade arquitetural.



5. Documentação Integrada

Solicite **documentação inline** como parte da geração:

- Comentários explicando **"porquês"** (não apenas "o quê")
 - **Exemplos de uso** para funções complexas
 - **Notas sobre decisões técnicas** tomadas
 - **Referências** para padrões ou algoritmos utilizados
-



6. Validação Automatizada como Pré-requisito

Antes de usar a IA, tenha preparado:

- **Testes unitários** que definem o comportamento esperado
- **Linters configurados** para padrões de código
- **Checagens de segurança** automatizadas
- **Pipeline de CI/CD** para validação contínua

Resultado: A IA gera código que já passa pelos seus critérios de qualidade.

7. Ciclo de Feedback Ativo

Use as **respostas da IA como aprendizado** para melhorar futuras solicitações:

- **Analise onde a IA errou** – falta de contexto? ambiguidade?
 - **Refine seu prompt** com base nos erros anteriores
 - **Crie templates de prompt** para tarefas recorrentes
 - **Compartilhe aprendizados** com a equipe
-

8. Complementaridade Humano-IA

Entenda as **fortalezas relativas**:

- **IA é excelente em:** sintaxe, padrões comuns, boilerplate, sugestões
- **Humano é essencial em:** design arquitetural, trade-offs de negócio, criatividade para soluções únicas, validação ética

Regra de ouro: Use a IA para **aumentar sua produtividade**, não para **substituir seu julgamento**.

9. Princípio da Responsabilidade Final

O desenvolvedor mantém a **responsabilidade integral** pelo código entregue:

- **Você responde** por bugs, vulnerabilidades e desempenho
 - **O código é seu** – a IA é apenas uma ferramenta
 - **Registro de decisões** deve ser mantido (por que escolheu essa abordagem?)
-

Fluxo Ideal de Trabalho

graph TD

A[Definir Requisitos Claros] --> B[Contextualizar para IA
Arquitetura + Frameworks]

B --> C[Gerar em Módulos Pequenos]

C --> D[Revisar Criticamente]

D --> E[Testar Imediatamente]
E --> F{Atende Padrões?}
F -->|Sim| G[Integrar e Documentar]
F -->|Não| H[Refinar Prompt e Iterar]
H --> C
G --> I[Feedback para Melhorias Futuras]

Armadilhas Comuns a Evitar

1. **Confiar cegamente** sem revisão
 2. **Prompt vago** = resultado imprevisível
 3. **Ignorar o contexto** do projeto existente
 4. **Esquecer de considerar** manutenibilidade futura
 5. **Negligenciar segurança** por pressa
-

Conclusão Essencial

A **qualidade do código gerado por IA é diretamente proporcional à qualidade do input humano**. Quanto mais você investe em **especificidade, contexto e processos de validação**, mais a IA se transforma de uma ferramenta geradora de código em um **parceiro de desenvolvimento verdadeiramente eficaz**.

A diferença entre código "funcional" e código "de produção" está na **aplicação consistente desses princípios** pelo desenvolvedor responsável.

</resposta1>

<resposta2>

Com certeza! Prepararei um template de prompt inicial para geração de código, focado nos princípios que discutimos. Você receberá um modelo prático e versátil para suas necessidades de desenvolvimento.

Com base nos princípios estabelecidos, aqui está um **template estruturado de prompt para geração de código** que você pode adaptar para qualquer tipo de software:

TEMPLATE DE PROMPT PARA GERAÇÃO DE CÓDIGO COM IA

Objetivo: Garantir que a IA gere código de alta qualidade, bem contextualizado e pronto para integração.

🎯 CONTEXTO DO PROJETO

[Descreva brevemente o sistema onde este código será integrado]

- **Arquitetura**: [MVC, Microserviços, Clean Architecture, etc.]
- **Frameworks/tecnologias principais**: [React 18, Spring Boot 3, Django 4, etc.]
- **Padrões de código da equipe**: [Convenções de nomenclatura, estrutura de pastas]
- **Estado atual**: [Novo recurso, refatoração, correção de bug]

📋 FUNCIONALIDADE ESPECÍFICA

[Descreva EXATAMENTE o que o código deve fazer]

- **Objetivo principal**:
- **Entradas esperadas**:
- **Saídas esperadas**:
- **Casos de uso principais**:
- **Casos de borda a considerar**:

⚙️ REQUISITOS TÉCNICOS

[Detalhe as especificações técnicas]

- **Linguagem/versão**: [Python 3.11, TypeScript 5.3, etc.]
- **Bibliotecas/disponíveis**: [Liste as que podem/deve usar]
- **Dependências externas**: [APIs, bancos de dados, serviços]
- **Restrições de performance**: [Tempo de resposta, uso de memória]

🛡️ RESTRIÇÕES E CONSIDERAÇÕES

[O que o código NÃO deve fazer ou deve evitar]

- **Segurança**: [Validações obrigatórias, sanitização de dados]
- **Performance**: [Otimizações necessárias, complexidade algorítmica]
- **Legibilidade**: [Nível de comentários, nomes descritivos]
- **Compatibilidade**: [Versões suportadas, navegadores, sistemas]

✅ EXPECTATIVAS DE QUALIDADE

[Como o código será avaliado]

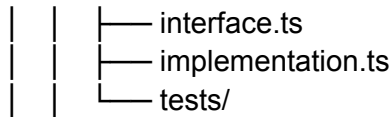
- **Testabilidade**: [Forneça exemplos de testes que devem passar]
- **Documentação**: [Comentários inline, documentação de funções]
- **Padrões a seguir**: [SOLID, DRY, design patterns específicos]
- **Integração**: [Como se conecta com código existente]

📝 EXEMPLO DE SAÍDA ESPERADA (OPCIONAL)

[Se aplicável, mostre o formato esperado]

``` Estrutura de arquivos:

```
|— src/
| |— module/
```



## EXEMPLO PRÁTICO PREENCHIDO

### ## 🎯 CONTEXTO DO PROJETO

Sistema de e-commerce em microsserviços, desenvolvendo módulo de carrinho de compras.

- **Arquitetura**: Microsserviços com comunicação via mensageria (RabbitMQ)
- **Frameworks/tecnologias principais**: Node.js 20, Express, TypeScript, MongoDB
- **Padrões de código da equipe**: CamelCase para funções, interfaces prefixadas com 'I'
- **Estado atual**: Novo recurso para cálculo de frete em tempo real

### ## 📋 FUNCIONALIDADE ESPECÍFICA

Serviço que calcula frete baseado em CEP, peso e dimensões dos produtos.

- **Objetivo principal**: Retornar opções de frete com prazos e valores
- **Entradas esperadas**: { cepDestino: string, produtos: Array<{ peso: number, altura: number, largura: number, comprimento: number }> }
- **Saídas esperadas**: { opcoes: Array<{ transportadora: string, prazo: number, valor: number }> }
- **Casos de uso principais**: Usuário adiciona produto ao carrinho, sistema calcula frete automaticamente
- **Casos de borda a considerar**: CEP inválido, produto com dimensões zero, múltiplos fornecedores

### ## ⚙️ REQUISITOS TÉCNICOS

- **Linguagem/versão**: TypeScript 5.3 com Node.js 20
- **Bibliotecas/disponíveis**: axios para HTTP, joi para validação
- **Dependências externas**: API dos Correios (<https://viacep.com.br/>), cache Redis
- **Restrições de performance**: Resposta em < 2 segundos, cache de 1 hora para CEPs

### ## 🛡️ RESTRIÇÕES E CONSIDERAÇÕES

- **Segurança**: Validar todos os inputs, sanitizar strings, evitar SQL injection
- **Performance**: Implementar cache eficiente, evitar chamadas HTTP desnecessárias
- **Legibilidade**: Comentar algoritmos complexos, nomes descritivos em inglês
- **Compatibilidade**: Node.js 18+, TypeScript estrito

### ## ✅ EXPECTATIVAS DE QUALIDADE

- **Testabilidade**: Fornecer testes unitários para casos normais e de borda
- **Documentação**: JSDoc para funções públicas, comentários explicando regras de negócio
- **Padrões a seguir**: Repository pattern para acesso a dados, injeção de dependências
- **Integração**: Interface compatível com serviço existente de carrinho (ICarrinhoService)



---

## COMO USAR ESTE TEMPLATE

1. **Preencha todas as seções** – quanto mais completo, melhor o resultado
  2. **Priorize clareza sobre brevidade** – seja explícito, não implícito
  3. **Inclua exemplos concretos** quando possível
  4. **Referencie código existente** se relevante para consistência
  5. **Especifique o NÃO desejado** tanto quanto o desejado
- 

## FLUXO RECOMENDADO

1. **Primeira iteração:** Use o template completo para funcionalidades complexas
  2. **Iterações seguintes:** Refira-se ao contexto já estabelecido, focando apenas nas mudanças
  3. **Refinamento contínuo:** Ajuste o template com base no que funciona para seu projeto
- 

## PRINCÍPIOS INCORPORADOS NO TEMPLATE

| Princípio        | Como o Template Aplica                  |
|------------------|-----------------------------------------|
| Especificidade   | Seções estruturadas forçam detalhamento |
| Contextualização | Contexto do projeto vem primeiro        |
| Modularidade     | Foca em uma funcionalidade por vez      |
| Validação        | Expectativas de qualidade explícitas    |
| Documentação     | Solicita comentários e estrutura clara  |

---

**Dica proativa:** Mantenha uma **biblioteca de prompts bem-sucedidos** – quando a IA gerar código excelente, salve o prompt usado como referência para tarefas similares futuras.

Este template transforma o ato de solicitar código de uma **pergunta vaga** em um **briefing profissional completo**, alinhando a IA com suas necessidades exatas desde o primeiro momento.

</resposta2>

</modelo3>

—

<modelo4>

<resposta1>

Excelente pergunta sobre os princípios para gerar código de alta qualidade com IA! Vou preparar uma resposta detalhada abordando as melhores práticas e considerações humanas essenciais para garantir a excelência no desenvolvimento assistido por IA.

## Princípios essenciais para gerar código de alta qualidade com apoio de IA

Quando um desenvolvedor usa IA para gerar código, o fator determinante da qualidade **não é apenas o modelo**, mas **como o humano conduz o processo**. Aqui estão os princípios fundamentais — independentes da linguagem, arquitetura ou tipo de software.

---

### 1. Clareza extrema no pedido

A IA só consegue gerar código de qualidade quando entende exatamente o que o humano quer.

Inclui:

- **Objetivo funcional claro**
- **Restrições** (performance, segurança, compatibilidade, requisitos legais)
- **Ambiente** (linguagem, framework, versão, plataforma)
- **Estilo desejado** (funcional, orientado a objetos, arquitetura hexagonal, etc.)

👉 Quanto mais ambíguo o pedido, maior a probabilidade de código ruim.

---

### 2. Contexto completo e atualizado

A IA precisa do mesmo contexto que um programador humano teria:

- Trechos existentes de código
- Arquitetura atual
- Dependências
- Padrões internos da empresa
- Requisitos de integração

- Formato da entrada e saída esperada

👉 Sem contexto, a IA inventa — não porque é “errada”, mas porque **completa lacunas**.

---

### 3. Dividir o problema em partes pequenas

A IA funciona melhor com granularidade clara.

Em vez de pedir “gere um módulo de pagamentos”, peça:

1. Interface
2. Modelos
3. Funções de validação
4. Conector com gateway
5. Testes unitários

👉 Pedidos menores → mais precisão → menos erros.

---

### 4. Fornecer exemplos do padrão esperado

A IA aprende com **demonstração**, não apenas explicação.

Inclua:

- Funções semelhantes já existentes
- Estrutura de pastas ideal
- Estilo de naming
- Exemplo de um teste “ideal”

👉 A IA tende a replicar exatamente o estilo demonstrado.

---

### 5. Solicitar raciocínio e não apenas código

Peça:

- Explicação das escolhas
- Justificativa técnica
- Comparação entre alternativas

- Identificação de riscos

Isso reduz escolhas equivocadas e ajuda você a validar.

---

## 6. Validar cuidadosamente cada saída

Nunca fazer “copiar e colar” automaticamente.

Sempre:

- Revisar lógica
- Executar testes
- Rodar análise estática
- Validar dependências
- Checar segurança (ex.: SQL injection, permissões, sanitização)

👉 IA acelera, mas **responsabilidade final é humana**.

---

## 7. Iteração guiada

O melhor resultado vem em ciclos:

1. Pedir algo pequeno
2. Testar
3. Ajustar
4. Reforçar estilo/padrões
5. Evoluir

👉 Trabalhar com IA é **colaborar**, não solicitar uma entrega final de primeira.

---

## 8. Fornecer feedback direto à IA

Exemplos:

- “Esse método está complexo, simplifique.”
- “Evite variáveis globais.”
- “Siga o padrão Repository.”
- “Reescreva usando programação funcional.”

👉 A IA melhora drasticamente com feedback explícito.

---

## 9. Priorizar segurança e boas práticas

Em cada pedido, lembrar a IA de:

- Sanitização de dados
- Princípio do menor privilégio
- Validação de entrada
- Logging e auditoria
- Gerenciamento seguro de segredos
- Tratamento de erros consistente

👉 Modelos de IA não priorizam segurança por padrão — você deve explicitamente pedir.

---

## 10. Pedindo testes, documentação e exemplos imediatamente

Código gerado pela IA é mais confiável quando inclui:

- Testes unitários e mocks
- Exemplos de uso
- Comentários objetivos
- Documentação da API

👉 Sem testes, você não tem segurança real sobre o código gerado.

---

## 11. Evitar pedir tudo de uma vez

Pedidos como “gere uma aplicação completa” levam a:

- Estruturas inconsistentes
- Arquiteturas artificiais
- Código não testado
- Conexões fictícias

👉 Arquiteturas grandes devem ser desenhadas em camadas, pedidas uma a uma.

---

## 12. Detectar e evitar alucinações de dependências

Sempre revisar se a IA:

- Criou libs inexistentes
- Usou APIs inexistentes
- Usou versões que não existem
- Referenciou métodos errados

👉 Peça explicitamente: **“Use apenas bibliotecas reais e confirme versões estáveis.”**

---

## 13. Documentar restrições antes de pedir código

Inclua:

- Tempo de execução desejado
- Limite de memória
- Restrições de escalabilidade
- Tolerância a falhas
- Compatibilidade com sistemas legados

👉 Código otimizado depende de restrições claras.

---

## 14. Alinhar expectativas sobre estilo e padrões da empresa

Se sua empresa usa:

- Clean Code
- DDD
- TDD
- SOLID
- KISS/YAGNI
- Arquitetura hexagonal
- Microserviços
- Monolito modular
- PSR (PHP), PEP8 (Python), ESLint (JS)

👉 Sempre informe antes. A IA seguirá fielmente.

---

## ■ Conclusão

**IA não substitui engenharia; ela acelera engenharia.  
Quanto melhor o humano guia o processo, melhor o código final.**

---

</resposta1>

<resposta2>

Excelente! Com base nos princípios que definimos, vou preparar um template de prompt inicial para a geração de código. Este template será versátil e aplicável a diferentes tipos de software.

## ■ Template de Prompt Inicial para Gerar Código com IA

Abaixo está um **template universal**, aplicável a **qualquer tipo de software**, linguagem ou arquitetura.

Ele incorpora todos os princípios essenciais para maximizar qualidade, precisão e segurança na geração de código por IA.

Você pode **copiar e colar**, preenchendo os campos entre colchetes.

---

## ■ TEMPLATE UNIVERSAL DE PROMPT INICIAL PARA GERAR CÓDIGO

### ■ Objetivo

Quero gerar código para **[descrever o que o código deve fazer]**.

### ■ Linguagem, stack e ambiente

Use a seguinte stack:

- Linguagem: **[ex.: Python/Node/Go/etc.]**
- Framework/bibliotecas obrigatórias: **[listar]**
- Versão da linguagem/framework: **[ex.: Python 3.11]**
- Ambiente-alvo: **[CLI, Web, API REST, Lambda, servidor, mobile etc.]**

⚠ **Use apenas bibliotecas reais e estáveis. Não invente dependências.**

## Contexto do sistema

Aqui está o contexto necessário:

- Arquitetura atual: **[descrever]**

Arquivos ou trechos relevantes:

[colar partes do código existente, se houver]

- 
- Padrões internos ou estilo de código usado: **[Clean Code, SOLID, DDD, FP etc.]**
- Restrições técnicas: **[ex.: sem estado global, compatível com X, sem acesso a rede etc.]**

## Requisitos funcionais

O código deve:

1. **[Requisito 1]**
2. **[Requisito 2]**
3. **[Requisito 3]**
4. **[Adicionar quantos forem necessários]**

## Requisitos não funcionais

Considere também:

- Segurança: **[sanitização, validações, permissões etc.]**
- Performance: **[limites, otimizações necessárias]**
- Escalabilidade: **[horizontal, vertical, ou não necessário]**
- Manutenibilidade: **[clareza, modularidade, desacoplamento etc.]**

## Estilo e qualidade esperada

Siga estas diretivas:

- Nomes claros e consistentes
- Funções pequenas, responsabilidade única
- Fluxo simples e legível
- Comentários apenas quando realmente necessários
- Evitar complexidade desnecessária (KISS / YAGNI)

## Extras obrigatórios

Inclua obrigatoriamente:



- **Testes unitários** completos para cada função criada
- **Exemplo de uso real** (snippet simples)
- **Explicação técnica das escolhas**
- **Tratamento robusto de erros**
- **Documentação mínima** (docstrings ou equivalente)

## ■ Antes de gerar o código

Resuma brevemente como pretende estruturar a solução e valide se sua interpretação corresponde ao que pedi.

Depois gere:

1. A explicação da abordagem
2. O código completo organizado
3. Os testes
4. O exemplo de uso

---

## ■ Versão resumida (para prompts rápidos)

Se quiser uma versão curta, aqui vai:

## ■ TEMPLATE CURTO

Gere código em **[linguagem]** para **[objetivo]**, considerando:

- Frameworks/bibliotecas: **[listar]**
- Requisitos funcionais: **[listar]**
- Requisitos não funcionais: **[listar]**
- Estilo: **[Clean Code, SOLID, funcional etc.]**
- Restrições: **[listar]**

Inclua:

- Testes unitários
- Documentação mínima
- Tratamento de erros
- Exemplo de uso

Antes de gerar, explique sua abordagem e confirme entendimento.

---

</resposta2>

</modelo4>

—  
<modelo5>  
<resposta1>

Excelente pergunta sobre os princípios para gerar código de alta qualidade com IA! Vou preparar uma resposta detalhada que aborda as melhores práticas e considerações essenciais para garantir a excelência do código, independentemente do tipo de software.

# Princípios Fundamentais para Geração de Código de Qualidade com IA

## 1. Especificidade e Contexto no Prompt

A qualidade do output é diretamente proporcional à qualidade do input. Prompts vagos geram código genérico e potencialmente inadequado.

**Práticas essenciais:**

- **Descreva o contexto completo:** arquitetura existente, linguagem, frameworks, padrões de projeto já utilizados
- **Especifique requisitos não-funcionais:** performance, escalabilidade, segurança, manutenibilidade
- **Inclua exemplos concretos:** dados de entrada/saída esperados, casos de uso específicos
- **Defina restrições técnicas:** versões de dependências, limitações de ambiente, padrões de código da equipe

👉 **Exemplo:** Em vez de "crie uma função de autenticação", use "crie uma função de autenticação JWT para Node.js 18+ usando bcrypt para hash de senhas, com rate limiting de 5 tentativas por minuto, seguindo padrões REST e retornando erros padronizados RFC 7807"

---

## 2. Revisão Crítica e Validação

**Nunca aceite código gerado cegamente.** A IA pode produzir código sintaticamente correto mas logicamente falho, inseguro ou ineficiente.

**Checklist de validação:**

-  **Segurança:** vulnerabilidades (SQL injection, XSS, CSRF), exposição de dados sensíveis, validação de entrada
  -  **Lógica de negócio:** edge cases, validações necessárias, tratamento de erros
  -  **Performance:** complexidade algorítmica, queries N+1, uso de memória
  -  **Manutenibilidade:** nomes descritivos, funções coesas, baixo acoplamento
  -  **Testes:** cobertura adequada, casos de borda contemplados
- 

### 3. Testes como Validação Obrigatória

**Código sem testes é código não confiável**, independente de quem o gerou.

**Estratégia de teste:**

- **Teste unitário:** para cada função/método gerado
- **Teste de integração:** para fluxos completos
- **Teste de edge cases:** valores nulos, strings vazias, números negativos, arrays vazios
- **Teste de segurança:** inputs maliciosos, limites de dados

 **Solicite testes junto com o código:** "gere a função X **com seus respectivos testes unitários usando Jest**, incluindo casos de erro"

---

### 4. Iteração Incremental

**Gere código em pequenas porções verificáveis**, não sistemas completos de uma vez.

**Abordagem recomendada:**

1. **Comece com interfaces/contratos:** defina assinaturas, tipos, schemas
2. **Implemente funcionalidades core:** lógica principal primeiro
3. **Adicione tratamento de erros:** depois da lógica funcionar
4. **Refatore e otimize:** quando tudo estiver testado

 **Evite:** pedir "crie um sistema completo de e-commerce" → alta probabilidade de código inconsistente e bugs difíceis de rastrear

---

### 5. Segurança por Design

**A IA pode não priorizar segurança** se não for explicitamente instruída.

**Sempre especifique:**

- **Sanitização de inputs:** validação, escape, type checking
- **Autenticação/autorização:** mecanismos apropriados ao contexto
- **Criptografia:** dados sensíveis em trânsito e em repouso
- **Secrets management:** nunca hardcode credenciais
- **Rate limiting:** proteção contra abuso

🔒 Inclua no prompt: "seguindo princípios OWASP Top 10" ou "com validação de entrada contra injeção"

---



## 6. Padrões e Convenções Consistentes

**Mantenha consistência** com a base de código existente.

**Forneça à IA:**

- **Style guides:** link para guia de estilo da equipe (Airbnb, Google, PEP 8)
- **Padrões de arquitetura:** MVC, Clean Architecture, Hexagonal
- **Nomenclatura:** convenções de nomes de variáveis, funções, classes
- **Estrutura de pastas:** organização do projeto

👉 "Gere seguindo o style guide do Airbnb e padrão Repository Pattern já utilizado no projeto"

---



## 7. Refatoração Consciente

**Use IA para refatorar, não apenas gerar novo código.**

**Casos de uso valiosos:**

- **Eliminação de code smells:** duplicação, funções muito longas, complexidade ciclomática alta
  - **Modernização:** migração para sintaxe mais recente, APIs atualizadas
  - **Performance:** otimização de algoritmos, queries
  - **Legibilidade:** renomeação, extração de métodos
- 



## 8. Documentação como Parte do Código

**Exija documentação junto com implementação.**

**Solicite:**

- **Comentários inline:** para lógica complexa ou não-óbvia
- **Docstrings/JSDoc:** para funções públicas (parâmetros, retorno, exceções)
- **README:** para módulos/bibliotecas
- **Exemplos de uso:** especialmente para APIs

👉 "Gere a função com JSDoc completo incluindo @param, @returns, @throws e exemplo de uso"

---

## 9. Performance e Otimização

**Especifique requisitos de performance** quando relevante.

**Considere:**

- **Complexidade algorítmica:**  $O(n)$  vs  $O(n^2)$  pode ser crítico
- **Uso de memória:** streams vs carregar tudo em memória
- **I/O:** paralelização, caching, lazy loading
- **Banco de dados:** índices, queries otimizadas, paginação

⚠️ A IA pode gerar soluções funcionais mas ineficientes para grandes volumes de dados

---

## 10. Tratamento de Erros Robusto

**Código de produção precisa lidar graciosamente com falhas.**

**Sempre inclua:**

- **Try-catch apropriado:** não silencie erros críticos
  - **Logging estruturado:** contexto suficiente para debugging
  - **Mensagens de erro úteis:** para desenvolvedores e usuários
  - **Fallbacks:** comportamento alternativo quando possível
  - **Validação de pré-condições:** fail fast quando entradas são inválidas
- 

## 11. Aprendizado e Melhoria Contínua

Trate código gerado por IA como oportunidade de aprendizado.

Práticas recomendadas:

- **Entenda cada linha:** se não entender, peça explicação ou refatore
  - **Compare abordagens:** peça múltiplas soluções e avalie trade-offs
  - **Questione decisões:** "por que usar X em vez de Y?"
  - **Documente padrões úteis:** crie biblioteca de prompts que funcionaram bem
- 



## 12. Versionamento e Rastreabilidade

Integre código gerado ao workflow normal de desenvolvimento.

Boas práticas:

- **Code review:** mesmo código de IA passa por revisão
  - **Commits atômicos:** não commite grandes blocos gerados de uma vez
  - **Mensagens descritivas:** documente que foi assistido por IA se relevante
  - **Testes em CI/CD:** validação automatizada antes de merge
- 



## Resumindo: Os 5 Mandamentos

1. 🔍 **Seja específico:** contexto rico = código melhor
  2. 🛠️ **Sempre teste:** código sem teste não existe
  3. 🛡️ **Segurança primeiro:** nunca assuma que está seguro
  4. 👁️ **Revise criticamente:** você é responsável pelo código, não a IA
  5. 📖 **Mantenha padrões:** consistência é mais valiosa que "código perfeito"
- 

**A IA é uma ferramenta poderosa, mas você continua sendo o engenheiro.** A responsabilidade final pela qualidade, segurança e manutenibilidade do código é **sempre humana**.

</resposta1>

<resposta2>

Com certeza! Prepararei um template de prompt inicial para geração de código, focado nos princípios que discutimos. Ele será versátil e aplicável a diferentes contextos de desenvolvimento de software.



# Template Universal para Geração de Código com IA

Aqui está um template estruturado que você pode adaptar para qualquer contexto de desenvolvimento. Preencha as seções relevantes e remova o que não se aplicar.

---



## Template de Prompt para Geração de Código

### ## CONTEXTO DO PROJETO

- **Linguagem/Framework**: [ex: Python 3.11 com FastAPI, React 18 com TypeScript, etc.]
- **Arquitetura**: [ex: Clean Architecture, MVC, Microserviços, Serverless]
- **Ambiente**: [ex: Node.js 20+, .NET 8, JVM 17]
- **Dependências principais**: [ex: PostgreSQL 15, Redis, AWS SDK]

### ## OBJETIVO

[Descreva em 2-3 frases o que precisa ser implementado]

Exemplo: "Implementar endpoint de autenticação de usuários que valide credenciais contra banco de dados, gere token JWT, e registre tentativas de login para auditoria."

### ## REQUISITOS FUNCIONAIS

- [Requisito 1: O que o código DEVE fazer]
- [Requisito 2: Fluxo esperado]
- [Requisito 3: Regras de negócio específicas]

**Entrada esperada:**

[Descreva formato de dados de entrada, com exemplo]

**Saída esperada:**

[Descreva formato de dados de saída, com exemplo]

### ## REQUISITOS NÃO-FUNCIONAIS

- **Performance**: [ex: resposta < 200ms, suportar 1000 req/s]
- **Segurança**: [ex: proteção contra SQL injection, validação de entrada, rate limiting de X tentativas por minuto]
- **Escalabilidade**: [ex: stateless, horizontalmente escalável]
- **Manutenibilidade**: [ex: código testável, baixo acoplamento]

### ## PADRÕES E CONVENÇÕES

- **\*\*Style guide\*\***: [ex: Airbnb JavaScript, PEP 8, Google Java Style Guide]
- **\*\*Padrões de código\*\***: [ex: Repository Pattern, Dependency Injection, Factory]
- **\*\*Nomenclatura\*\***: [ex: camelCase para variáveis, PascalCase para classes]
- **\*\*Estrutura\*\***: [ex: seguir estrutura de pastas /services, /controllers, /models]

## ## TRATAMENTO DE ERROS

- [Como lidar com erros: exceções customizadas, códigos HTTP específicos]
- [Logging: estruturado, níveis (info/warn/error), informações a incluir]
- [Validações necessárias antes de processar]

## ## SEGURANÇA

- [Checklist: sanitização de inputs, autenticação/autorização necessária]
- [Dados sensíveis: como tratar (criptografia, não logar, etc.)]
- [Conformidade: OWASP Top 10, LGPD/GDPR se aplicável]

## ## TESTES

Gere também:

- **\*\*Testes unitários\*\*** usando [framework: Jest, pytest, JUnit, xUnit]
- **\*\*Casos de teste obrigatórios\*\***:
  - Caso de sucesso (happy path)
  - Casos de erro [especificar quais]
  - Edge cases [valores nulos, vazios, limites, etc.]
- **\*\*Cobertura mínima\*\***: [ex: 80%]

## ## DOCUMENTAÇÃO

Inclua:

- [ ] Comentários inline para lógica complexa
- [ ] Docstrings/JSDoc completos (parâmetros, retorno, exceções)
- [ ] Exemplo de uso da função/classe
- [ ] README se for módulo/biblioteca

## ## RESTRIÇÕES E CONSIDERAÇÕES

- [Limitações técnicas: versões específicas, bibliotecas que NÃO podem ser usadas]
- [Constraints de infraestrutura: limites de memória, timeout]
- [Compatibilidade: precisa funcionar com código legado específico]

## ## FORMATO DE SAÍDA

Por favor, gere:

1. **\*\*Código principal\*\*** (bem estruturado e comentado)
2. **\*\*Testes\*\*** (com casos descritos acima)
3. **\*\*Documentação\*\*** (conforme especificado)
4. **\*\*Dependências\*\*** (listar novas dependências necessárias)

---



**\*\*Nota\*\***: Priorize [escolha: segurança/performance/legibilidade] nesta implementação.

---



## Versão Resumida (Quick Start)

Para situações mais simples, use esta versão condensada:

**\*\*Contexto\*\***: [Linguagem/Framework + Arquitetura]

**\*\*Objetivo\*\***: [O que implementar em 1-2 frases]

**\*\*Entrada/Saída\*\***:

- Input: [formato e exemplo]
- Output: [formato e exemplo]

**\*\*Requisitos críticos\*\***:

- Segurança: [validações necessárias]
- Performance: [se relevante]
- Padrões: [style guide ou padrão de projeto]

**\*\*Gere\*\***:

1. Código implementando [funcionalidade]
2. Testes unitários com [framework]
3. Documentação (JSDoc/docstrings)

**\*\*Casos de teste obrigatórios\*\***: happy path, [erro 1], [erro 2], [edge case]

---



## Exemplo Preenchido (Autenticação)

### ## CONTEXTO DO PROJETO

- **\*\*Linguagem/Framework\*\***: Node.js 20 com Express.js e TypeScript
- **\*\*Arquitetura\*\***: Clean Architecture com camadas controller/service/repository
- **\*\*Ambiente\*\***: Node.js 20.x, Docker
- **\*\*Dependências principais\*\***: PostgreSQL 15, Redis para cache, bcrypt, jsonwebtoken

### ## OBJETIVO

Implementar serviço de autenticação de usuários que valide credenciais contra PostgreSQL, gere token JWT com expiração de 24h, implemente rate limiting de 5 tentativas por IP a cada 15 minutos, e registre todas tentativas de login (sucesso e falha) para auditoria.

## ## REQUISITOS FUNCIONAIS

- Validar email e senha fornecidos pelo usuário
- Verificar senha usando bcrypt com hash armazenado no banco
- Gerar token JWT contendo userId, email e roles
- Incrementar contador de tentativas falhas (reset após sucesso)
- Bloquear conta após 5 tentativas falhas consecutivas
- Registrar log de auditoria com timestamp, IP, user-agent, resultado

**\*\*Entrada esperada:\*\***

```
``json
{
 "email": "user@example.com",
 "password": "senha123"
}
```

**Saída esperada (sucesso):**

```
{
 "token": "eyJhbGciOiJIUzI1NiIs...\"",
 "expiresIn": 86400,
 "user": {
 "id": "uuid",
 "email": "user@example.com",
 "roles": ["user"]
 }
}
```

## REQUISITOS NÃO-FUNCIONAIS

- **Performance:** Resposta < 300ms (incluindo consulta DB)
- **Segurança:** Proteção contra timing attacks, inputs sanitizados, senhas nunca logadas, tokens seguros
- **Escalabilidade:** Stateless, rate limiting via Redis distribuído
- **Manutenibilidade:** Código testável, dependency injection, baixo acoplamento

## PADRÕES E CONVENÇÕES

- **Style guide:** Airbnb TypeScript Style Guide
- **Padrões de código:** Service Layer Pattern, Repository Pattern, Dependency Injection via constructor

- **Nomenclatura:** camelCase para variáveis/funções, PascalCase para classes/interfaces, UPPER\_SNAKE\_CASE para constantes
- **Estrutura:** /src/controllers, /src/services, /src/repositories, /src/middlewares

## TRATAMENTO DE ERROS

- Exceções customizadas: UnauthorizedError, TooManyAttemptsError, ValidationError
- Códigos HTTP: 401 para credenciais inválidas, 429 para rate limit, 422 para validação
- Logging estruturado (Winston) com níveis apropriados, incluindo requestId, userId (se disponível), mas NUNCA senhas
- Validações: email formato válido, senha não-vazia, sanitização contra SQL injection

## SEGURANÇA

- Sanitizar todos inputs (validação de email, escape de strings)
- Rate limiting: 5 tentativas por IP em janela de 15min (via Redis)
- Usar bcrypt.compare() com timing constante
- Tokens JWT assinados com HS256 e secret do environment
- Não expor se email existe ou se senha está incorreta (mensagem genérica)
- Conformidade com OWASP Top 10 (A01, A02, A03, A07)

## TESTES

Gere também:

- **Testes unitários** usando Jest com TypeScript
- **Casos de teste obrigatórios:**
  -  Login com credenciais válidas
  -  Login com senha incorreta
  -  Login com email não cadastrado
  -  Login com email inválido (formato)
  -  Login após exceder rate limit
  -  Login com conta bloqueada
  -  Reset de contador após login bem-sucedido
  -  Registro de auditoria em todos os casos
- **Cobertura mínima:** 90%
- **Mocks:** repositório de usuários, Redis, logger

## DOCUMENTAÇÃO

Inclua:

- Comentários inline para lógica de rate limiting e bloqueio
- JSDoc completos com @param, @returns, @throws
- Exemplo de uso do AuthService
- Descrição das exceções customizadas

## RESTRIÇÕES E CONSIDERAÇÕES

- Não usar bibliotecas de autenticação prontas (Passport.js) - implementação custom
- Secret do JWT deve vir de variável de ambiente JWT\_SECRET
- Timeout de conexão com Redis: 5s com fallback gracioso
- Compatibilizar com middleware de logging existente que injeta requestId

## FORMATO DE SAÍDA

Por favor, gere:

1. **AuthService** (classe com método authenticate)
2. **AuthController** (endpoint POST /auth/login)
3. **Exceções customizadas** (UnauthorizedError, etc.)
4. **Testes unitários** (AuthService.test.ts com todos os casos)
5. **Interfaces TypeScript** (LoginRequest, LoginResponse, etc.)
6. **Lista de dependências** necessárias (package.json additions)

---

**Nota:** Priorize **segurança** nesta implementação, garantindo proteção contra timing attacks e rate limiting robusto.

---

##  **\*\*Como Usar o Template\*\***

#### **\*\*Passo 1\*\*:** Copie o template

Escolha a versão completa ou resumida conforme a complexidade

#### **\*\*Passo 2\*\*:** Preencha as seções

- Seja específico nos exemplos de entrada/saída
- Liste casos de erro que você já antecipa
- Mencione padrões que já existem no projeto

#### **\*\*Passo 3\*\*:** Remova seções irrelevantes

Se não há requisitos de performance específicos, remova a seção

### \*\*Passo 4\*\*: Itere

Comece pedindo apenas o código principal, depois peça os testes, depois refatorações

### \*\*Passo 5\*\*: Salve prompts que funcionaram

Crie biblioteca pessoal de prompts efetivos para reutilizar

---

## 📌 \*\*Dicas de Uso\*\*

- ✓ \*\*Faça\*\*: Adapte o template para seu contexto específico
- ✓ \*\*Faça\*\*: Seja concreto com exemplos de dados reais
- ✓ \*\*Faça\*\*: Especifique frameworks e versões exatas
- ✓ \*\*Faça\*\*: Mencione padrões já usados no projeto

- ✗ \*\*Evite\*\*: Preencher tudo se não for necessário
- ✗ \*\*Evite\*\*: Ser vago ("faça algo performático")
- ✗ \*\*Evite\*\*: Pedir sistema completo de uma vez
- ✗ \*\*Evite\*\*: Esquecer de especificar testes

---

Este template garante que você forneça **contexto rico e estruturado** para a IA, resultando em código de **maior qualidade, mais seguro e mais alinhado** com suas necessidades reais!



</resposta2>

</modelo5>

—

<modelo6>

<resposta1>

Excelente pergunta sobre os princípios para gerar código de alta qualidade com IA! Vou preparar uma resposta detalhada que aborda as melhores práticas e considerações humanas essenciais para garantir a excelência do software, independentemente do seu tipo.

Vou pesquisar fontes acadêmicas e confiáveis sobre desenvolvimento assistido por IA e melhores práticas para garantir qualidade no código gerado. Vou buscar informações adicionais sobre práticas específicas de engenharia de prompts e qualidade de código. Com base nas pesquisas acadêmicas e publicações técnicas do MITRE e OpenAI, aqui estão os **princípios fundamentais** que um humano deve considerar ao gerar código com IA para garantir a melhor qualidade:

---

## 🎯 1. Preparação e Entendimento do Problema

## Compreenda profundamente antes de solicitar código

- Desenvolva especificações claras e arquitetura antes de usar a IA
- Defina casos de borda, entradas/saídas esperadas e modos de falha
- Crie pseudocódigo ou diagramas de fluxo manualmente antes de delegar
- A IA acelera a implementação, mas **não substitui o entendimento do domínio**

**Por que importa:** Estudos mostram que desenvolvedores usando IA sem preparação adequada frequentemente geram código que "parece" correto mas viola convenções do domínio ou introduz bugs sutis.

---



## 2. Engenharia de Contexto (Prompts Estratégicos)

### Forneça contexto rico e estruturado

- Alimente a IA com: regras de negócio, convenções do projeto, exemplos de código existente
- Especifique padrões de design, estilo de código e restrições técnicas
- Use arquivos de contexto externos (como AGENTS.md) para manter consistência
- Divida problemas complexos em subtarefas menores e bem definidas

**Evite:** prompts vagos como "melhore meu código" — seja específico: "extraia a lógica de validação em uma função separada"

---



## 3. Desenvolvimento Orientado a Testes (TDD)

### Testes são especificações executáveis

- **Regra de Ouro:** Escreva testes ANTES de pedir implementação à IA
- Forneça casos de teste concretos como especificações comportamentais
- Valide que novos testes falhem ANTES de implementar a funcionalidade
- Solicite à IA que identifique casos de borda, condições extremas e erros

**⚠ Cuidado:** A IA pode gerar "testes de papel" — testes que passam superficialmente mas não validam a lógica real. Sempre revise testes gerados.

---



## 4. Monitoramento Ativo e Iteração

## Nunca delegue e abandone

- Monitore o progresso da IA continuamente
  - Pergunte-se: "Ela está alterando coisas que não deveria? Ignorando minhas solicitações? Introduzindo novos problemas?"
  - **Reinicie quando necessário:** Se a conversa ficar confusa com muitas tentativas falhas, comece fresh com lições aprendidas
  - Mantenha bom controle de versão para reverter facilmente
- 

## 5. Revisão Crítica e Ceticismo Saudável

### Desconfie das afirmações de sucesso da IA

- A IA frequentemente afirma sucesso mesmo quando não resolveu o problema
- Leia e compreenda o código gerado — não aceite cegamente
- Verifique se a solução faz sentido para seu domínio específico
- Alinhe com suas expectativas prévias de como o problema deveria ser resolvido

### Pergunte-se:

- Isso resolve o problema de forma metodologicamente correta?
  - Há violações de padrões do domínio?
  - O código é apropriado cientificamente/técnicamente?
- 

## 6. Refinamento Incremental com Objetivos Claros

### Melhere por partes, não tudo de uma vez

- Resista à tentação de pedir "melhore minha base de código"
  - Aborde refinamentos incrementalmente: performance, legibilidade, tratamento de erros, modularidade
  - Especifique o objetivo exato de cada iteração
  - Teste exaustivamente após cada mudança — a IA pode quebrar código funcional
- 

## 7. Segurança e Validação de Dependências

### Verifique o que a IA sugere

- Revise todas as dependências e bibliotecas sugeridas
- Valide licenças de código gerado (problemas de copyright)
- Verifique vulnerabilidades de segurança em código importado
- Não aceite código que você não entende completamente sem revisão

---

## 8. Parceria Human-IA (Modelo "Delegar-Revisar-Possuir")

Adote o framework de três camadas:

| Papel   | Responsabilidade                                                                                   |
|---------|----------------------------------------------------------------------------------------------------|
| Delegar | IA faz a primeira passada: scaffolding, lógica CRUD, testes iniciais                               |
| Revisar | Humano avalia escolhas de design, performance, segurança, alinhamento de domínio                   |
| Possuir | Humano mantém controle de abstrações arquiteturais, requisitos ambíguos, trade-offs de longo prazo |

---

## 9. Documentação e Reprodutibilidade

Mantenha rastreabilidade

- Documente que ferramentas de IA foram usadas
- Registre abordagens de prompting e procedimentos de validação
- Mantenha expectativas claras sobre testes e reprodutibilidade
- Crie documentação que evolua com o código (a IA pode ajudar a manter atualizado)

---

## 10. Consciência das Limitações

Entenda o que a IA faz bem vs. mal

 A IA é boa em:

- Gerar boilerplate e código repetitivo



- Sugerir casos de teste e edge cases
- Identificar oportunidades de refatoração
- Criar infraestrutura de testes automatizados

### ✗ A IA é ruim em:

- Garantir correção lógica sem supervisão
- Entender nuances de domínio específico
- Tomar decisões arquiteturais de longo prazo
- Validar que o código atende a requisitos científicos/métodos rigorosos

---

## Fontes Consultadas

- [Ten Simple Rules for AI-Assisted Coding in Science](#) — ArXiv, 2025
- [Building an AI-Native Engineering Team](#) — OpenAI
- [Software Engineering with Generative AI Tools](#) — MITRE, 2024
- [Is AI the Better Programming Partner?](#) — Carnegie Mellon University

---

**Resumindo:** O valor da IA não está em substituir o julgamento humano, mas em **amplificar a capacidade do desenvolvedor** de focar no que realmente importa: correção, coerência, manutenibilidade e qualidade de longo prazo. A qualidade do código gerado por IA é diretamente proporcional à qualidade da preparação, especificação e revisão humana.

</resposta1>

<resposta2>

Entendido! Prepararei um template de prompt inicial focado em gerar código, seguindo os princípios que definimos. Você receberá uma estrutura clara e versátil para suas necessidades de codificação.

Aqui está um **template de prompt inicial robusto**, estruturado para maximizar a qualidade do código gerado pela IA:

---

## TEMPLATE DE PROMPT PARA GERAÇÃO DE CÓDIGO

##  CONTEXTO DO PROJETO

**\*\*Domínio da Aplicação:\*\*** [Descreva o domínio: e-commerce, saúde, financeiro, jogos, etc.]

**\*\*Stack Tecnológico:\*\***

- Linguagem: [Ex: Python 3.11, TypeScript, Java, etc.]
- Framework: [Ex: React, Django, Spring Boot, etc.]
- Bibliotecas relevantes: [Listar principais dependências já utilizadas]

**\*\*Arquitetura/Padrões:\*\***

- [Ex: Clean Architecture, MVC, Microservices, Serverless]
- [Padrões de design: Repository, Factory, Strategy, etc.]

---

**## 🎯 PROBLEMA A SER RESOLVIDO**

**\*\*Descrição Funcional:\*\***

[Descreva O QUE o código deve fazer em linguagem de negócio. Seja específico sobre o comportamento esperado.]

**\*\*Entradas Esperadas:\*\***

- [Tipo e formato de cada parâmetro de entrada]
- [Restrições de validação: ranges, formatos, obrigatoriedade]

**\*\*Saídas Esperadas:\*\***

- [Tipo de retorno]
- [Estrutura do objeto/response]
- [Códigos de erro/estados de falha]

**\*\*Fluxo de Processamento:\*\***

1. [Passo 1 do algoritmo]
2. [Passo 2 do algoritmo]
3. [Passo 3 do algoritmo]

---

**## 🧪 CASOS DE TESTE (Especificações Executáveis)**

**#### Caso de Sucesso Padrão**

- **\*\*Entrada:\*\*** [dados concretos]
- **\*\*Saída Esperada:\*\*** [resultado exato esperado]

**#### Casos de Borda (Edge Cases)**

- [Cenário extremo 1: valores mínimos/máximos, strings vazias, nulls]
- [Cenário extremo 2: entradas malformadas, caracteres especiais]
- [Cenário extremo 3: concorrência, volumes grandes, timeouts]

**#### Casos de Erro**

- [Erro 1: entrada inválida → comportamento esperado]
- [Erro 2: dependência indisponível → comportamento esperado]
- [Erro 3: timeout/limites excedidos → comportamento esperado]

---

## ## 📁 REQUISITOS TÉCNICOS E PADRÕES

### ### Estilo de Código

- [Convenção de nomenclatura: camelCase, snake\_case, PascalCase]
- [Máximo de linhas por função/método: ex: 30 linhas]
- [Complexidade ciclomática máxima: ex: 10]
- [Regras de formatação: uso de linters, prettier, black, etc.]

### ### Qualidade e Manutenibilidade

- ☒ Código deve ser autoexplicativo (nomes claros de variáveis/funções)
- ☒ Comentários apenas quando necessário (explicar o "porquê", não o "o quê")
- ☒ Funções devem ter responsabilidade única (SRP)
- ☒ Evitar código duplicado (DRY)
- ☒ Tratamento explícito de erros (não silenciar exceções)
- ☒ Logs apropriados para debugging e monitoramento

### ### Performance e Segurança

- [Restrições de tempo de execução:  $O(n)$ ,  $O(\log n)$ , etc.]
- [Considerações de memória: streaming vs. carregar tudo em memória]
- [Validação de entradas: sanitização, prevenção de injeção]
- [Não expor dados sensíveis em logs ou erros]

---

## ## 📁 ESTRUTURA ESPERADA

**\*\*Localização do Arquivo:\*\*** `[caminho/do/arquivo.extensão]`

**\*\*Interface Pública:\*\***

``typescript

// Exemplo de assinatura esperada

function nomeDaFuncao(param1: Tipo1, param2: Tipo2): TipoRetorno

### **Dependências Internas:**

- [Módulos/classes que este código deve importar/utilizar]
- [Contratos/interfaces que deve implementar]

### Arquivos Relacionados:

- [Testes: `__tests__/nome.test.ts`]
  - [Types: `types/nome.types.ts`]
  - [Utils auxiliares: `utils/helper.ts`]
- 

## ENTREGÁVEIS ESPERADOS

1. **Código fonte** implementando a funcionalidade descrita
  2. **Testes unitários** cobrindo:
    - Caso de sucesso padrão
    - Todos os edge cases listados
    - Todos os cenários de erro
  3. **Documentação inline** (JSDoc, docstrings, etc.) explicando:
    - Propósito da função/classe
    - Parâmetros e retornos
    - Exceções que pode lançar
  4. **Exemplo de uso** em código (snippet demonstrando como utilizar)
- 

## RESTRIÇÕES E ATENÇÕES ESPECIAIS

- [NÃO fazer X: ex: não usar bibliotecas externas não aprovadas]
  - [EVITAR: ex: evitar mutação de estado global]
  - [GARANTIR: ex: garantir thread-safety se aplicável]
  - [VERIFICAR: ex: verificar compatibilidade com versão X da dependência Y]
- 

## CONTEXTO ADICIONAL (Opcional)

### Código Existente Relacionado:

// Cole aqui código relevante do projeto para manter consistência  
// Ex: interfaces, tipos, funções auxiliares já existentes

**Exemplo de Implementação Similar no Projeto:** [Caminho ou snippet de código similar já existente que serve como referência de estilo]

---

## ## 💡 INSTRUÇÕES DE USO DO TEMPLATE

### ### Como utilizar:

1. **\*\*Preencha TODAS as seções obrigatórias\*\*** antes de solicitar código à IA
2. **\*\*Seja específico nas entradas/saídas\*\*** — dados concretos são melhores que descrições vagas
3. **\*\*Forneça pelo menos 3 casos de teste\*\*** (sucesso + 2 edge cases)
4. **\*\*Inclua código existente\*\*** quando quiser manter consistência com a base atual
5. **\*\*Revise as restrições\*\*** para evitar que a IA sugira abordagens inadequadas

### ### Checklist antes de enviar:

- [ ] Descrevi o domínio e contexto de negócio?
- [ ] Especifiquei entradas e saídas com tipos?
- [ ] Liste casos de borda e erros esperados?
- [ ] Defini padrões de código e qualidade?
- [ ] Indiquei onde o arquivo deve ser criado?
- [ ] Especifiquei o que deve ser entregue (código + testes + docs)?

---

Este template segue o princípio de que **\*\*a qualidade da saída é proporcional à qualidade da especificação de entrada\*\***. Quanto mais contexto e restrições você fornecer, mais preciso e adequado será o código gerado.

</resposta2>

</modelo6>

</respostas-modelos>